

SE212 Database System and Design

Lab 9: Joins — querying more than one table (20 points)

Name _____ SID _____

In Lab 8 you queried one table at a time. Real questions almost always need two or more tables: *which customer placed this order?*, *what is in it?*, *which department does this employee work in?* This lab practises the join types from Lecture 9:

- **INNER JOIN** — only rows that match in both tables
- **LEFT JOIN** — every row of the left table, matched or not
- **Self join** — a table joined to itself
- Joins combined with **GROUP BY**, **ORDER BY** and **LIMIT**

You use the Murach sample databases you installed in Lab 8: **om** (order management) and **ex** (examples). If they are missing, re-run `create_databases.sql` from Lab 8 (see *Before the lab* below). Save every statement you write in one file named `Lab9_<SID>.sql` so you can re-run it.

Before the lab

1. Start MySQL Server and open MySQL Workbench (Lab 8, Option A or B).
2. Check the two databases are there. Run:

```
USE om; SHOW TABLES; -- customers, items, order_details, orders
USE ex; SHOW TABLES; -- 11 tables, you need departments, employees, projects
```

3. If `om` or `ex` is missing, run `mysql -u root -p < create_databases.sql` from the `Lab8_murach` folder, or open the script in Workbench and execute it, exactly as in Lab 8 Part 2.
4. Keep the ER diagram `ER-diagram-OMdatabase-Murach.pdf` open. You need the foreign keys to write every join.

Rules of the road. Give every table an alias (`customers c`, `orders o`) and qualify every column (`c.customer_id`). Write `JOIN ... ON` (the explicit form). The old comma form (`FROM a, b WHERE ...`) is accepted only where the task says so.

Part 1: om — order management (12 points)

The `om` database has four tables. Keys are underlined in the ER diagram.

Table	Columns you need	Foreign key
customers	<u>customer_id</u> , customer_first_name, customer_last_name, customer_state	

Table	Columns you need	Foreign key
orders	order_id, customer_id, order_date, shipped_date	customer_id → customers
order_details	order_id, item_id, order_qty	order_id → orders, item_id → items
items	item_id, title, artist, unit_price	

1.1 (2 points) Two tables. List every order with the name of the customer who placed it: `order_id`, `order_date`, `customer_last_name`, `customer_first_name`. Newest order first. How many rows come back, and which customer placed the newest order?

Rows returned: _____ Newest order placed by: _____

1.2 (2 points) Order contents. Order **829** is the newest order. Show what is in it: `title`, `artist`, `order_qty`, `unit_price`, and a calculated column `line_total = order_qty * unit_price`. Then add up the line totals by hand (or with SUM) and write the order total.

Order total: _____

1.3 (3 points) Four tables. Show everything customer **Randall** has ever bought: `customer_last_name`, `order_id`, `order_date`, `title`, `order_qty`. Order by `order_id` then `title`. You must join all four tables in one statement. Take a screenshot (**S1**) of the result grid.

Rows returned: _____

1.4 (3 points) Join + GROUP BY. Which items earn the most money? For every item show `title`, `units_sold` (total quantity ordered) and `revenue` (total of `order_qty * unit_price`). Show the three highest revenues first.

Top item and its revenue: _____

1.5 (2 points) Outer join. One order in `om` has no order lines at all. Find it with a `LEFT JOIN` from `orders` to `order_details` and an `IS NULL` test. Give `order_id` and `order_date`.

Then answer in a comment: `orders INNER JOIN order_details` returns 68 rows. How many rows does `orders LEFT JOIN order_details` return, and why is it different?

Part 2: ex — employees and departments (8 points)

Three tables in `ex`. They have **no foreign key constraints**, which is why the data can be inconsistent on purpose.

Table	Columns
departments	department_number, department_name
employees	employee_id, last_name, first_name, department_number, manager_id
projects	project_number, employee_id

2.1 (2 points) Inner join. List `employee_id`, `last_name`, `first_name` and `department_name` for every employee, ordered by `employee_id`, using an `INNER JOIN`. There are 9 employees but the query returns fewer rows. Which employees are missing, and why? Look at their `department_number`.

Rows returned: _____ Missing: _____

2.2 (2 points) Left join. Change 2.1 so that every employee appears, with `department_name` shown as NULL when the department does not exist. Which join keyword did you change and which table must be on the left?

2.3 (2 points) The other direction. Which department has **no employees**? Write a query that returns only that department's number and name. Hint: start FROM `departments`, or use RIGHT JOIN.

2.4 (2 points) Self join. Every employee's `manager_id` is the `employee_id` of another row in the same table. Show `employee_id`, the employee's `last_name` as `employee`, and the manager's `last_name` as `manager`, for all 9 employees. The company president has no manager and must still appear, with `manager = NULL`. Take a screenshot (**S2**).

Submission checklist (read this before you upload)

Submit on Mango (CMU Canvas), assignment **Lab 9**, before the deadline shown there. This is individual work. Upload exactly two files, named with your student ID:

File 1: Lab9_<SID>.sql. One plain-text SQL script that the TA can run from top to bottom without any error, in this order and with these comment headers:

```
-- Lab 9 | Name: <your name> | SID: <your SID>
```

USE om;

```
-- ===== 1.1 Orders with customer =====
```

```
-- SELECT here
```

```
-- rows: <number> newest order by: <last name>
```

```
-- ===== 1.2 Order 829 contents =====
```

```
-- SELECT here
```

```
-- order total: <amount>
```

```
-- ===== 1.3 Everything Randall bought =====
```

```
-- SELECT here
```

```
-- rows: <number>
```

```
-- ===== 1.4 Revenue per item =====
```

```
-- SELECT here
```

```
-- top item: <title>, <revenue>
```

```
-- ===== 1.5 Order with no lines =====
```

```
-- SELECT here
```

```
-- LEFT JOIN rows: <number> because: <one sentence>
```

USE ex;

```
-- ===== 2.1 Inner join =====
```

```
-- SELECT here
```

```
-- rows: <number> missing: <names>, because: <one sentence>
```

```
-- ===== 2.2 Left join =====
```

```
-- SELECT here
-- changed: <keyword>, left table: <table>

-- ===== 2.3 Department with no employees =====
-- SELECT here

-- ===== 2.4 Self join =====
-- SELECT here
```

File 2: Lab9_<SID>_screenshots.pdf (or two PNG files S1.png, S2.png). Each screenshot must show the whole Workbench window, not a cropped grid, so the connection name and the query are visible.

Screenshot	What it must show	From task
S1	Result grid of 1.3 (Randall's purchases) with the query above it	1.3
S2	Result grid of 2.4 (employee and manager) with 9 rows	2.4

How you will be graded

Item	The TA checks	Evidence	Points
1.1	Correct JOIN ... ON, four columns, ORDER BY date DESC; 47 rows and newest customer stated	File 1	2
1.2	JOIN to items, calculated <code>line_total</code> , correct order total	File 1	2
1.3	All four tables joined on the right keys, WHERE on last name, ORDER BY; row count	File 1 + S1	3
1.4	JOIN + GROUP BY item, SUM for both columns, ORDER BY revenue DESC, LIMIT 3	File 1	3
1.5	LEFT JOIN + IS NULL finds the order; row-count question answered	File 1	2
2.1	INNER JOIN, correct row count, missing employees explained	File 1	2
2.2	LEFT JOIN with employees on the left; keyword and table named	File 1	2
2.3	Returns exactly one department	File 1	2
2.4	Self join with two aliases, LEFT JOIN so the president appears	File 1 + S2	2
	Total		20

Deductions. Script stops with an error: the TA grades only the parts that ran before the error. Missing alias or unqualified column that MySQL rejects as *ambiguous*: that task scores 0 until fixed. Wrong file names or missing SID: -1. Missing screenshot: 0 for that item. Submitted after the deadline: as per course syllabus.